DAY 5 OF 7

Admin & user screens

The complete UI: admin manages all content and users, users take timed quizzes and see scores

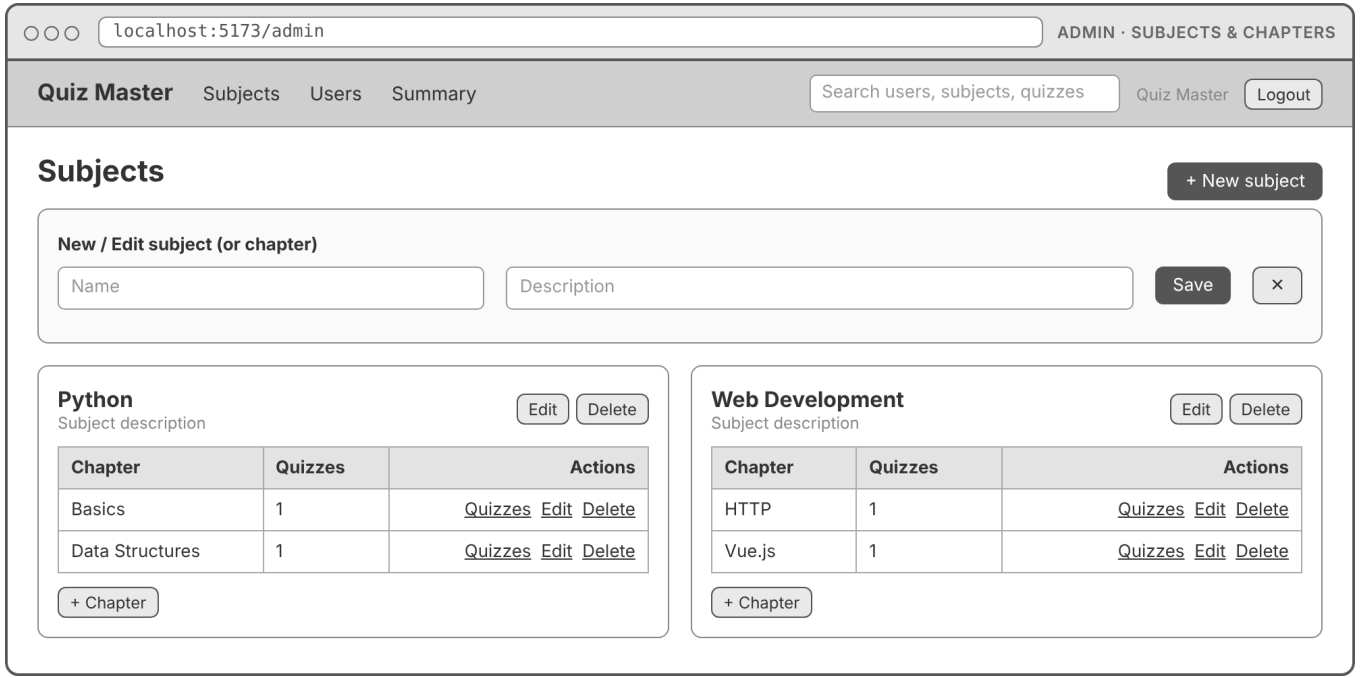
Today's plan

10 min	Recap: store, router, api.js
15 min	The page pattern: data → mounted → api
30 min	Admin: subjects, quizzes, questions
15 min	Admin: users and search
35 min	User: quiz list, timed quiz, scores
15 min	Full demo, wrap-up

By the end you can

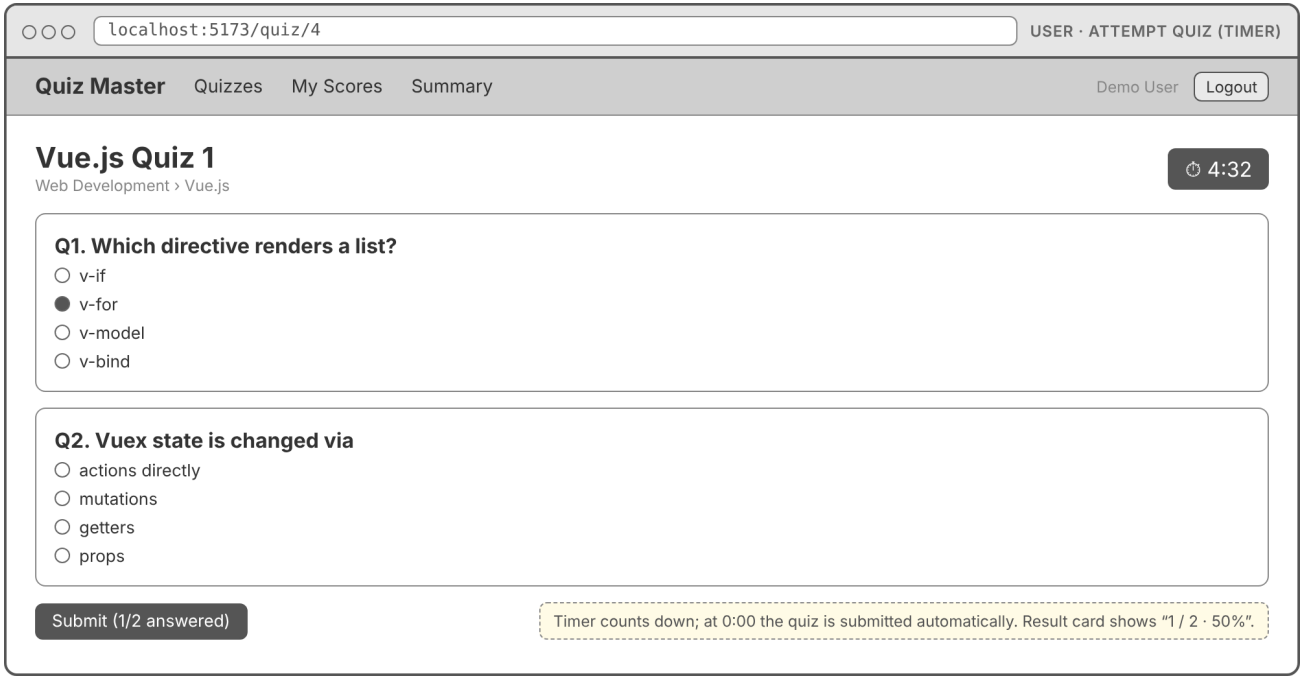
- Build CRUD screens with v-for, v-if and v-model
- Load each page’s data in mounted()
- Use route params, query params and watchers
- Build a countdown timer and clean it up

Admin home: subjects and chapters



Each subject is a card with its chapters. One small form adds or edits a subject or a chapter.

User: attempt a quiz



All questions on one page, a countdown timer, auto-submit at 0:00, then the score card.

All screens built today

URL	Role	Shows
`/admin`	admin	Subject cards with chapter tables, add/edit form
`/admin/chapters/:id`	admin	Quizzes of a chapter: title, date, duration, remarks
`/admin/quizzes/:id`	admin	Questions; a radio button marks the correct option
`/admin/users · /admin/search?q=`	admin	Block / unblock users; search results
`/dashboard`	user	All quizzes, subject filter, Start button
`/quiz/:id · /scores`	user	Timed quiz and result; past attempts

Store or page?

Vuex store (from Day 4)

- Data many pages need at once
- Who is logged in: token, user
- The message on screen
- Unchanged today

The page's data()

- Data one page shows
- Subjects, quizzes, questions, scores
- The form being edited
- Loaded in mounted() with api.get

Every page follows one pattern

```
export default {
  data() {
    return {
      chapterId: this.$route.params.id,   // from the URL
      quizzes: [],
      form: null,                         // null = form hidden
    }
  },
  mounted() {
    this.loadQuizzes()                    // 1. load on open
  },
  methods: {
    async loadQuizzes() {
      const response = await api.get(
        '/chapters/' + this.chapterId + '/quizzes')
      this.quizzes = response.data.quizzes  // 2. table updates
    },
  },
}
```



data()

start with empty values



mounted()

ask Flask for the data when the page opens



v-for

the table redraws by itself when quizzes changes

One form for add and edit

```
newQuiz()      { this.form = { title: '', time_duration: 10 } },
editQuiz(quiz) { this.form = { ...quiz } },     // a copy

async save() {
  try {
    if (this.form.id) {
      await api.put('/quizzes/' + this.form.id, this.form)
    } else {
      await api.post('/chapters/' + this.chapterId + '/quizzes',
                     this.form)
    }
    this.form = null
    this.$store.dispatch('showMessage',
      { type: 'success', text: 'Quiz saved' })
    this.loadQuizzes()                // 3. reload
  } catch (error) {
    this.$store.dispatch('showMessage',
      { type: 'danger', text: errorText(error) })
  }
},
```



Copy, then edit

{ …quiz } so the table does not change while typing



id or no id

id → PUT (edit), no id → POST (new)



Reload

call loadQuizzes() again after saving

A countdown timer that cleans up

```
async mounted() {
  const url = '/quizzes/' + this.quizId + '/attempt'
  const response = await api.get(url)
  this.quiz = response.data.quiz
  this.questions = response.data.questions
  this.secondsLeft = this.quiz.time_duration * 60
  this.startTimer()
},
unmounted() {
  clearInterval(this.timer)       // leaving the page: stop it
},
methods: {
  startTimer() {
    this.timer = setInterval(() => {      // every 1000 ms
      this.secondsLeft = this.secondsLeft - 1
      if (this.secondsLeft === 0) this.submit()   // time is up
    }, 1000)
  },
},
```



setInterval

ticks every second



Auto-submit

at 0 the quiz is sent



unmounted()

stop the timer when leaving the page

Search: watch the URL

frontend/src/views/admin/SearchView.vue

```
export default {
  data() { return { users: [], subjects: [], quizzes: [] } },
  mounted() { this.search() },            // first search
  watch: {
    '$route.query.q'() { this.search() },      // ?q= changed
  },
  methods: {
    async search() {
      const q = this.$route.query.q
      const response = await api.get('/admin/search',
                                     { params: { q } })
      this.users = response.data.users
      this.subjects = response.data.subjects
      this.quizzes = response.data.quizzes
    },
  },
}
```



Navbar search

goes to /admin/search?q=…



Watcher

the page stays open, only ?q= changes – so watch it



Same method

mounted() and the watcher both call search()

Styled with Bootstrap only

No custom CSS file – every look comes from Bootstrap classes installed with npm.



Layout

container, row and col grid, d-flex, gap



Components

card, table, badge, breadcrumb, alert, form-control



Mobile

navbar-expand-md with a Vue toggle, table-responsive

Demo checklist

```
Terminal

python seed_demo.py
python app.py              # backend
npm run dev                # frontend

Admin:  admin@quizmaster.com / admin123
User:   user@quizmaster.com  / user123
```

Check that...

- Admin adds subject → chapter → quiz → 2 questions
- Search "python" from the navbar
- Block the user → login shows "blocked"
- User filters, starts a quiz, timer auto-submits
- Phone width: navbar collapses, tables scroll

Recap

- Every page: data() → mounted() → api.get → v-for table
- The store did not change today – it still only knows who is logged in
- One form object handles both add (POST) and edit (PUT)
- Route params, query params and watchers drive the pages
- The timer starts in mounted() and stops in unmounted()

Next: Day 6 – background jobs with Celery and Redis: CSV export, reminders, monthly report.